



CPT316: Programming Language Implementation and Paradigms
SCHOOL OF COMPUTER SCIENCES UNIVERSITI SAINS MALAYSIA

Semester I,

Academic Session: 2025/2026

Group Number : 29

Assignment 2

Dr. Azman Bin Ab Malik

Name	Matric No	USM Email Address	Contribution
AHMED M.H. TAHA	22304588	ahmedtaha@student.usm.my	Python scripting and Node-RED integratio
ZHOU YIANG	22305072	evanzhou@student.usm.my	Technical report writing and documentation
ABDULLAH ABDULRAHMAN SALEM OMAR AL- BAKER	2230585	abdullahalbaker@student.usm.my	Node-RED integration and Python scripting
AL Rasyidiqy Muhammad Khathir	165293	khathir@student.usm.my	Final report compilation and formatting

Submission Date: 3 January 2026

Table of Contents

1.Introduction.....	3
2.System Overview	3
3.Programming Paradigms Analysis	3
3.1Python Flask Backend (Imperative/Object-Oriented Paradigm)	3
3.2Node-RED Frontend (Flow-Based/Dataflow Paradigm)	4
4.Implementation Details	5
4.1Python Flask API Implementation.....	5
4.2Node-RED Dashboard Implementation	6
5.Inter-Platform Communication	6
6.Paradigm Comparison and Analysis.....	8
7.Challenges and Solutions	9
Conclusion	9
References	10

1. Introduction

This report presents the design and implementation of an Energy Monitor System developed. The project demonstrates the integration of two distinct programming paradigms Python Flask representing the imperative and object-oriented paradigm for backend services, and Node-RED representing the flow-based (dataflow) paradigm for frontend visualization and user interaction.

The Energy Monitor System is designed to simulate a smart home energy monitoring solution that tracks power consumption across multiple household devices, calculates electricity costs based on Malaysian TNB rates, and provides real-time visualization through an interactive dashboard. This project showcases how different programming paradigms can be effectively combined to create a cohesive and functional application.

2. System Overview

The Energy Monitor System consists of two main components that communicate via HTTP APIs:

Table 1: System Components, Technologies, and Their Roles

Component	Technology	Paradigm	Role
Backend API	Python Flask	Imperative/OOP	Data management, business logic, API endpoints
Frontend Dashboard	Node-RED	Flow-Based/Dataflow	UI visualization, user interaction, data display

The system simulates six household devices: Air Conditioner (2000W), Refrigerator (150W), TV (100W), Washing Machine (500W), Lights (60W), and Computer (20W). Users can monitor real-time power consumption, toggle devices on/off, view cost estimates, and track power trends over time.

3. Programming Paradigms Analysis

3.1 Python Flask Backend (Imperative/Object-Oriented Paradigm)

Python, used with the Flask web framework, represents the **imperative** and **object-oriented** programming paradigms. In imperative programming, the developer explicitly specifies the sequence of operations to be performed, focusing on *how* to achieve a result through step-by-step instructions.

Key Characteristics of the Imperative Paradigm in Our Implementation:

- **Sequential Execution:** The Flask routes execute code line by line, performing calculations and data manipulations in a defined order.

- **State Management:** The global 'devices' list maintains application state, with functions explicitly modifying this state.
- **Explicit Control Flow:** Loops and conditionals (for, if statements) control program execution explicitly.
- **Mutable Data:** Device states are modified directly through assignment operations.

Code Example - Toggle Device Function:

```
@app.route('/api/toggle/<int:device_id>', methods=['GET', 'POST']) def
toggle_device(device_id):    for device in devices:        if device['id'] ==
device_id:
        device['status'] = 'inactive' if device['status'] == 'active' else
'active'        return jsonify({'success': True, 'device': device})
```

3.2 Node-RED Frontend (Flow-Based/Dataflow Paradigm)

Node-RED represents the **flow-based** (or **dataflow**) programming paradigm. In this paradigm, programs are constructed as directed graphs where nodes represent operations and edges represent data flow between operations. This paradigm focuses on *what* data transformations occur rather than the explicit sequence of operations.

Key Characteristics of the Flow-Based Paradigm in Our Implementation:

- **Visual Programming:** The entire application logic is represented as connected nodes in a visual flow editor.
- **Data-Driven Execution:** Nodes execute when data (messages) arrive at their inputs, not based on explicit sequencing.
- **Implicit Parallelism:** Multiple data flows can execute concurrently without explicit threading code.
- **Declarative Connections:** Wires between nodes declare data dependencies rather than control flow.
- **Event-Driven Architecture:** Inject nodes trigger flows based on time intervals or events.

Table 2: Layered Architecture of the Proposed System

Layer	Technology	Responsibility
Presentation	Node-RED Dashboard	User interface, data visualization, user interaction
Application	Node-RED Flows	Data flow orchestration, API calls, data transformation
Business Logic	Flask Routes	Cost calculation, consumption tracking, device management
Data	Python Lists	Device state storage, in-memory data persistence

4. Implementation Details

4.1 Python Flask API Implementation

The Python Flask backend serves as the data management and business logic layer. It provides five main API endpoints that handle device management and energy calculations.

Table 3: Backend API Endpoints and Their Functionalities

Endpoint	Method	Description
/api/devices	GET	Returns all devices with current status and total power
/api/consumption	GET	Calculates current power consumption and daily kWh
/api/cost	GET	Computes daily, monthly, yearly costs using TNB rates
/api/toggle/<id>	GET/POST	Toggles a specific device on or off
/api/statistics	GET	Returns overall system statistics and efficiency

Key Implementation Features:

- **CORS Support:** Flask-CORS enables cross-origin requests from Node-RED dashboard.
- **JSON Responses:** All endpoints return JSON-formatted data for easy parsing.
- **Malaysian TNB Rates:** Cost calculations use RM 0.57 per kWh rate.
- **Realistic Simulation:** Random variation ($\pm 50W$) added for realistic power readings.
- **Timestamp Inclusion:** All responses include ISO-formatted timestamps.

4.2 Node-RED Dashboard Implementation

The Node-RED frontend consists of multiple interconnected flows that handle data retrieval, transformation, and visualization. The dashboard is organized into four main UI groups.

Table 4: Dashboard UI Components and Refresh Rates

UI Group	Components	Update Interval
Power Monitoring	Power gauge, Active devices count, Daily kWh display	5 seconds
Cost Analysis	Daily cost display, Monthly cost display	5 seconds
Device Control	Interactive device cards with toggle buttons	3 seconds
Power Trend	Line chart showing power consumption over time	60 seconds

Node Types Used in the Implementation:

- **Inject Nodes:** Trigger periodic data refresh at configurable intervals.
- **HTTP Request Nodes:** Make GET/POST requests to Flask API endpoints.
- **Function Nodes:** Extract and transform data from API responses.
- **UI Template Node:** Custom HTML/CSS/Angular for device control panel.
- **UI Gauge:** Visual power consumption indicator (0-3000W range).
- **UI Chart:** Time-series visualization of power trends.
- **UI Text:** Display formatted text values for costs and statistics.
- **Delay Node:** Adds 300ms delay after toggle for state synchronization.

5. Inter-Platform Communication

The communication between Node-RED (JavaScript) and Python Flask demonstrates seamless integration between different programming paradigms through standardized HTTP/JSON protocols.

Communication Flow:

1. Node-RED Inject Node triggers at interval (e.g., every 5 seconds)
2. HTTP Request Node sends GET request to Flask endpoint
3. Flask processes request, executes business logic
4. Flask returns JSON response with data
5. Function Node extracts specific fields from response

6. UI Node displays data to user

Example Data Flow - Get Consumption:

Node-RED Request: GET http://localhost:5000/api/consumption

Flask Response:

```
{  
  "current_power_watts": 2210,  
  "daily_consumption_kwh": 53.04, "timestamp":  
  "2026-01-01T14:30:00"  
}
```

Function Node Extraction:

```
msg.payload = msg.payload.current_power_watts; return  
msg;
```

Toggle Device Interaction:

The device control panel demonstrates bidirectional communication. When a user clicks a toggle button, the UI Template node sends the device ID, which triggers an HTTP request to the Flask toggle endpoint. The Flask server updates the device state and returns confirmation. A 300ms delay ensures the state change propagates before refreshing the display.

The Energy Monitor System provides comprehensive energy monitoring capabilities:

Table 5: Functional Features and Their Implementation

Feature	Description	Implementation
Real-time Monitoring	Live power consumption display	Gauge widget with 5s refresh
Device Control	Toggle devices on/off	Interactive buttons with API calls
Cost Estimation	TNB-based cost calculation	Daily/Monthly/Yearly projections
Power Trends	Historical consumption chart	Line chart with 20-min rolling window
Device Statistics	Active/inactive device count	Calculated from device status
Efficiency Metrics	Power usage percentage	Current vs. max potential power

6. Paradigm Comparison and Analysis

This section compares the two paradigms used in our implementation, analyzing their strengths, weaknesses, and suitability for different aspects of the application.

Table 6: Comparative Analysis of Imperative and Flow-Based Development Approaches

Aspect	Python/Flask (Imperative)	Node-RED (Flow-Based)
Code Structure	Sequential, text-based	Visual, graph-based
Learning Curve	Moderate (requires coding)	Low (drag-and-drop)
Debugging	Traditional debugging tools	Visual flow inspection
Reusability	Functions and modules	Subflows and node reuse
Performance	Generally faster execution	Overhead from runtime
Flexibility	Complete control over logic	Limited by available nodes
Maintenance	Code versioning (Git)	JSON-based flow export
Best For	Complex business logic	Rapid prototyping, IoT

Analysis and Justification:

The combination of Python Flask and Node-RED proves highly effective for this IoT monitoring application. Python Flask excels at handling the business logic, calculations, and data management where precise control is needed. The imperative nature allows explicit handling of edge cases and complex calculations like cost projections.

Node-RED complements this by providing rapid dashboard development without extensive frontend coding. The flow-based paradigm naturally maps to the event-driven nature of IoT systems where data flows from sensors through processing to displays. The visual programming approach also makes the system easier to understand and modify for non-programmers.

7. Challenges and Solutions

Table 7: Identified Challenges and Corresponding Solutions

Challenge	Solution
CORS blocking requests	Added Flask-CORS extension with CORS(app)
State sync after toggle	Implemented 300ms delay before refresh
Real-time data updates	Used inject nodes with configurable intervals
Custom device control UI	Created HTML/CSS template with Angular bindings
API response parsing	Function nodes to extract specific JSON fields
Dashboard layout	Organized into logical UI groups with proper sizing

Conclusion

This project successfully demonstrates the integration of two distinct programming paradigms to create a functional Energy Monitor System. The Python Flask backend, representing the imperative/object-oriented paradigm, handles data management and business logic with precision and flexibility. The Node-RED frontend, representing the flow-based/dataflow paradigm, provides an intuitive visual programming environment for building interactive dashboards.

The key learning outcomes from this project include:

- Understanding the fundamental differences between imperative and flow-based paradigms
- Practical experience in designing RESTful APIs for inter-system communication
- Appreciation for how different paradigms can complement each other
- Skills in rapid IoT dashboard prototyping using Node-RED
- Knowledge of real-world energy monitoring system design

The project demonstrates that selecting the right programming paradigm for each component leads to more maintainable, efficient, and effective solutions. The combination of Python for backend logic and Node-RED for frontend visualization represents a powerful approach for IoT and monitoring applications.

References

- [1] Flask Documentation. (2024). Flask Web Development. <https://flask.palletsprojects.com/>
- [2] Node-RED Documentation. (2024). Node-RED: Low-code programming for event-driven applications. <https://nodered.org/docs/>
- [3] Tenaga Nasional Berhad. (2024). TNB Tariff Rates. <https://www.tnb.com.my/>
- [4] Van Roy, P., & Haridi, S. (2004). Concepts, Techniques, and Models of Computer Programming. MIT Press.
- [5] Morrison, J. P. (2010). Flow-Based Programming: A New Approach to Application Development. CreateSpace.
- [6] Python Software Foundation. (2024). Python Documentation. <https://docs.python.org/>
- [7] Nick Samoylov, Learn Java 17 Programming: Learn the fundamentals of Java Programming with this updated guide with the latest features, 2nd Edition, 2022.
- [8] . David Amos, Dan Bader, Joanna Jablonski, Fletcher Heisler, Python Basics: A Practical Introduction to Python 3, Revised and Updated 4th Edition, 2020
- [9] Terrence W. Pratt & Marvin V. Zelkowitz, Programming Languages: Design & Implementation, 4th Edition, Prentice Hall, 2001"